Test documentation: FaceAttend EDU

Jonattan Steven Rizo Solano

Diego Andrés Gutiérrez Núñez

Juan David Arboleda Perdomo

Centro de la Industria, la Empresa y los Servicios, Servicio Nacional de Aprendizaje

Technologist in Software Analysis and Development, Cohort 3145556

November 4, 2026

Contents

Test plan 3
- Purpose and scope 3
- Test levels 3
- Environments 3
- Entry and exit criteria 4
- Responsibilities 4

Test cases 5
- End-to-end cases 5
- Unit cases 6

Results and evidence 7
- Continuous integration 7
- End-to-end flows 7
- Full functional review 8
- Limitations 8

Unit test catalog 9
- Summary 9
- Introduction 9
- Units Under Test 9
- Conventions Applied in this Module 10
- Catalog of Documented Unit Tests 10
- Test Doubles and Data 12
- Coverage and Exclusions 13
- Test → ERF → PA Traceability 13
- Flaky Tests and Quarantine 14
- Link to Quality Actions 14
- Conclusions 14
- References 14

Test documentation: FaceAttend EDU

Test plan

Purpose and scope

This plan defines how FaceAttend EDU is verified before each production release. It covers the nine microservices, the shared authentication library, the web and mobile applications, the API contract and the critical MVP flows.

Load tests and tests with the physical fingerprint reader are out of scope, because they are not part of the automated flows that run in continuous integration.

Test levels

Table 1

Test levels

Level	Goal	Tool	When it runs
Unit and integration per service	Business rules, use cases and adapters	JUnit and Maven, Jest, pytest	On every pull request and before each deployment
Web and mobile interface	Components, view models and navigation	Jest with snapshots	On every pull request and before each deployment
API contract	Detect differences between code and OpenAPI	`openapi / drift` job	On every pull request and before each deployment
End to end	The 6 critical flows on the full system in Docker Compose	`tools/e2e/mvp-flujos.mjs`	`E2E flujos MVP` workflow
Full functional review	Endpoints, screens, buttons and coherence between web, API and database	`verificar-estado.mjs` and `verificar-contrato.mjs`	On demand, before a closing milestone

Environments

Table 2

Environments

Environment	Use	Data
GitHub Actions	Unit, integration and contract tests	Test doubles and disposable data

Environment	Use	Data
Docker Compose in CI	E2E flows, with a random-secret `.env` per run	Synthetic data generated on each run
Local with Docker Compose	Full functional review	Seeded test data

Entry and exit criteria

- **Entry:** the code builds and the pull request targets the integration branch.
- **Exit:** every CI job finishes green. The production deployment depends on that result, because `Deploy production` runs CI first.
- **Skipped tests:** accepted only when explicitly marked in the code. They are reported in the results table.

Responsibilities

The three team members write the tests of the module they develop. Execution is automatic in GitHub Actions, and the final deployment approval is manual in the `production` environment.

Test cases

End-to-end cases

Each case runs against the full system. The detail is in `tools/e2e/mvp/flujos.mjs`.

Table 3

End-to-end cases

ID	Flow	Verified step	Expected result
E2E-01	Sign-in	First sign-in with the initial password	The gateway blocks writes with 403 `PASSWORD_CHANGE_REQUIRED`
E2E-02	Sign-in	Initial password change	The old password returns 401, the new one signs in and can write
E2E-03	Sign-in	New account with a wrong password	401, and the password policy is applied
E2E-04	School onboarding	Public sign-up of school and administrator	Stays pending activation
E2E-05	School onboarding	The Super Administrator activates the school	The school administrator can sign in
E2E-06	Academic structure	Create program, period, group, course and room	The records are created
E2E-07	Academic structure	Create instructor and students	Each one signs in and accepts the terms
E2E-08	Academic structure	Enroll students in the group	Both students are enrolled
E2E-09	Scheduling	Second block in the same room and time	Rejected
E2E-10	Scheduling	Create a class session	Starts as Open, unique per block and date
E2E-11	Attendance	The instructor marks Present manually	Recorded, and a duplicate is rejected
E2E-12	Attendance	The instructor closes the session	`closedBy` and `closedAt` are recorded
E2E-13	Attendance	Closing with a student without a record	That student becomes Absent and the present one does not change
E2E-14	Justification	Approve without the required attachment	Rejected
E2E-15	Justification	The student uploads the PDF and the administrator approves	`reviewedBy` and `reviewedAt` are recorded
E2E-16	Web session	Web sign-in	HttpOnly cookie and no session id in the body

ID	Flow	Verified step	Expected result
E2E-17	Web session	Write without the anti-CSRF header	Returns 403
E2E-18	Web session	Sign-out	The session stops working across the gateway

Unit cases

The catalog of documented unit cases, with its traceability to requirements, is in the annex of this document.

Results and evidence

Continuous integration

- **Tested version:** `v1.7.0`, commit `07119ab7`, branch `main`.
- **Run date:** November 4, 2026.
- **Result:** all 13 jobs finished green.
- **Evidence:** CI run on GitHub Actions.

Table 4

Continuous integration

Component	Technology	Total	Passed	Failed	Skipped
01-ms-identity	Java / Maven	442	428	0	14
02-ms-authorization	Java / Maven	142	142	0	0
04-ms-scheduling	Java / Maven	145	144	0	1
05-ms-attendance	Java / Maven	193	192	0	1
shared / auth	Java / Maven	19	19	0	0
03-ms-academic	TypeScript / Jest	126	126	0	0
07-ms-configuration	TypeScript / Jest	70	70	0	0
08-ms-notification	TypeScript / Jest	201	201	0	0
09-ms-quality	TypeScript / Jest	159	159	0	0
06-ms-biometric	Python / pytest	213	213	0	0
Web	Expo / Jest	998	909	0	89
Mobile	Expo / Jest	614	614	0	0
Total		**3322**	**3217**	**0**	**105**

The `openapi / drift` job also finished green. It is not in the table because it compares contracts and does not report a test count.

The totals come from the logs of that run. For the Java services, the passed column subtracts the skipped tests from the total Maven reports.

End-to-end flows

- **Workflow:** `E2E flujos MVP`.
- **Tested commit:** `b67a5933`, from November 2, 2026.
- **Result:** finished green.

- **Evidence:** E2E run on GitHub Actions.

This run predates `v1.7.0`. It is the most recent E2E run on `main` at the time of this delivery.

Full functional review

Run locally on October 31, 2026, with `verificar-estado.mjs`.

Table 5

Full functional review

Reviewed item	Total	Passed	Failed	Warnings	Skipped
API endpoints	108	86	0	0	22
Web screens	49	43	0	0	6
Web buttons	53	39	0	4	10
Mobile screens	27	27	0	0	0

The coherence check reviewed 44 points and 8,755 records in 24 lists, with no findings. The contract review of November 1, 2026 passed all 88 operations compared between the web and the database.

Limitations

- These two local reviews predate `v1.7.0` and were not repeated for this delivery.
- Skipped tests provide no evidence for the cases they cover: 16 in Java services and 89 on the web.
- The four buttons with warnings remain as review items.

Unit test catalog

Summary

This set of unit tests protects the transactional and security core of FaceAttendEDU. It focuses on validating the correct implementation of the RBAC model, the robustness of the authentication flow (including account lockout and session rotation), the prevention of overlaps in academic scheduling, and the integrity of attendance records. Additionally, it mitigates critical security risks through the validation of liveness logic and access control in the biometric service.

Keywords: JUnit 5, Pytest, Mockito, RBAC, Biometrics, Traceability.

Introduction

The FaceAttendEDU back-end is built using a Hexagonal Architecture, which allows the isolation of business logic into independent Use Cases. Due to the criticality of identity management and attendance control, it is imperative to have a unit testing suite that validates each business rule before integration. These tests ensure that future changes in the infrastructure do not degrade the basic functionality of the system.

Document Objective

To prescribe and document the execution of the unit tests for the back-end modules, linking them directly to functional requirements and technical risks.

Document Scope

This documents all unit tests for the services: `ms-identity`, `ms-authorization`, `ms-scheduling`, `ms-attendance`, and `ms-biometric`. Integration tests, load tests, and User Acceptance Tests (UAT) are outside the scope of this document.

Units Under Test

Table 6

Units Under Test

Unit	Responsibility	Associated ERF	Associated PA	Mitigated ANA Risk
`AuthenticateUserUseCaseImpl`	Login and lockout management	ERF 1.3	PA-SEG-01	Brute force / Unauthorized access

Unit	Responsibility	Associated ERF	Associated PA	Mitigated ANA Risk
`RefreshSessionUseCaseImpl`	Opaque session rotation	ERF 1.3	PA-SEG-02	Session theft / Replay attack
`CheckPermissionUseCase`	RBAC permission evaluation	ERF 1.1.2	PA-SEG-03	Privilege escalation
`CreateScheduleBlockUseCase`	Overlap validation	ERF 2.3.2	PA-OPS-01	Schedule conflicts
`AttendanceUniqueness`	Prevent duplicate records	ERF 3.1	PA-DAT-01	Attendance inconsistency
`FacialImageEndpoints`	Liveness and enrollment flow	ERF 3.1.1	PA-BIO-01	Facial impersonation
`SecurityGuard (Biometric)`	API role validation	ERF 1.1.2	PA-SEG-03	Unauthorized access to biometrics

Conventions Applied in this Module

The **AAA (Arrange-Act-Assert)** structure and the **Given-When-Then** pattern are used for scenario writing. Identifier language is English, while result documentation is in English. `@Tags` are implemented in JUnit to categorize tests by requirement.

Catalog of Documented Unit Tests

01-ms-identity: Authentication and Security

AuthenticateUserUseCaseImpl - Successful Login

- **Traceability Block:** `@requirement(ERF_1_3) @tags(Security, HappyPath)`
- **Test Code:**

```
@Test
void shouldStartSessionWhenCredentialsAreValid() {
    // Arrange
    var user = new User("username", "hashed_pass");
    when(userRepository.findByUsername("username")).thenReturn(Optional.of(user));

    // Act
    var result = authenticateUseCase.authenticate("username", "password");

    // Assert
    assertThat(result.getSessionId()).isNotNull();
    verify(sessionRepository).save(any());
}
```

- **Expected Result:** The system validates credentials and returns a `UserSessionDto` with an active session ID.

- **Edge Case Covered:** Not applicable (happy path).

AuthenticateUserUseCaseImpl - Lockout due to Failed Attempts

- **Traceability Block:** `@requirement(ERF_1_3) @tags(Security, EdgeCase)`
- **Test Code:**

```
@Test
void shouldLockAccountAfterFiveFailedAttempts() {
    // Arrange
    when(loginAttemptRepository.getCount("user1")).thenReturn(5);

    // Act & Assert
    assertThatThrownBy(() -> authenticateUseCase.authenticate("user1", "wrong_pass"))
        .isInstanceOf(AccountLockedException.class)
        .hasMessageContaining("blocked for 30 minutes");
}
```

- **Expected Result:** The system throws an account locked exception upon reaching the limit of 5 attempts.
- **Edge Case Covered:** Attempt to access an already locked account.

02-ms-authorization: Access Control (RBAC)

CheckPermissionUseCase - Permission Verification

- **Traceability Block:** `@requirement(ERF_1_1_2) @tags(RBAC, HappyPath)`
- **Test Code:**

```
@Test
void shouldAllowAccessWhenUserHasPermissionViaRole() {
    // Arrange
    var user = new User(UUID.randomUUID());
    var role = new Role("Admin", List.of("user:create"));

when(userRoleRepository.findRolesByUserId(user.getId())).thenReturn(List.of(role));

    // Act
    boolean allowed = checkPermissionUseCase.hasPermission(user.getId(),
"user:create");

    // Assert
    assertThat(allowed).isTrue();
}
```

- **Expected Result:** The system returns `true` if the user possesses the permission through any of their assigned roles.
- **Edge Case Covered:** User with multiple roles where only one grants the permission.

04-ms-scheduling: Academic Scheduling

CreateScheduleBlockUseCase - Overlap Prevention

- **Traceability Block:** `@requirement(ERF_2_3_2) @tags(Scheduling, EdgeCase)`
- **Test Code:**

```
@Test
void shouldThrowExceptionWhenRoomIsBusy() {
    // Arrange
    var newBlock = new ScheduleBlock(roomA, instructorB, timeRange);
    when(overlapGuard.isRoomBusy(roomA, timeRange)).thenReturn(true);

    // Act & Assert
    assertThatThrownBy(() -> createUseCase.execute(newBlock))
        .isInstanceOf(ScheduleOverlapException.class);
}
```

- **Expected Result:** The system prevents the creation of a time block if the room is already occupied in that range.
- **Edge Case Covered:** Partial overlap of schedules (start in the middle of another block).

06-ms-biometric: Facial Processing

Facial Image Endpoints - Liveness Failure

- **Traceability Block:** `@requirement(ERF_3_1_1) @tags(Biometrics, Security)`
- **Test Code:**

```
def test_identify_fails_when_liveness_is_rejected():
    # Arrange
    client = TestClient(app)
    payload = {"image": "base64_photo_of_screen", "token": "valid_token"}

    # Act
    response = client.post("/biometric/identify", json=payload)

    # Assert
    assert response.status_code == 400
    assert "Liveness check failed" in response.json()["error"]
```

- **Expected Result:** The system rejects identification if the liveness algorithm detects it is a photograph or screen.
- **Edge Case Covered:** Presentation attack (Spoofing).

Test Doubles and Data

Table 7

Test Doubles and Data

Double	Isolated Dependency	Type (stub/mock/spy/fake)	Reason for Isolation	Assumed Contract
`UserRepository`	PostgreSQL DB	Mock (Mockito)	Avoid infrastructure dependency in logic tests	Return `Optional<User>`
`SessionRepository`	Redis / DB	Fake (InMemory)	Execution speed and predictable state	Key-value storage
`LivenessService`	Biometric API	Mock (Monkeypatch)	Avoid costly image processing in unit tests	Boolean (Pass/Fail)

Test Data: Random UUID generators and fixed seeds for biometric vectors are used in Python tests to ensure reproducibility. No real user data is used.

Coverage and Exclusions

Table 8

Coverage and Exclusions

Excluded Element	Technical Reason	Actual Coverage at Other Level	Responsible
`GlobalExceptionHandler`	Logic delegated to Spring framework	Integration Tests (API)	Lead Dev
`Kafka Producers`	External cluster dependency	System Tests (E2E)	DevOps

Coverage achieved:

- Lines: 88%
- Branches: 82%

Test → ERF → PA Traceability

Table 9

Test → ERF → PA Traceability

Test	ERF	PA	ANA Risk	Status
`AuthenticateUserUseCaseImplTest`	ERF 1.3	PA-SEG-01	Brute force	PASS
`RefreshSessionUseCaseImplTest`	ERF 1.3	PA-SEG-02	Session theft	PASS
`CheckPermissionUseCaseImplTest`	ERF 1.1.2	PA-SEG-03	Privilege escalation	PASS
`CreateScheduleBlockUseCaseImplTest`	ERF 2.3.2	PA-OPS-01	Schedule conflicts	PASS
`FacialImageEndpoints_Liveness`	ERF 3.1.1	PA-BIO-01	Facial impersonation	PASS

Flaky Tests and Quarantine

No flaky tests have been identified in the current suite. All tests are deterministic.

Link to Quality Actions

A preventive action has been opened in `QMS-SW-CAPA-001` to implement mutation testing in the biometric module, as the complexity of vectors requires more rigorous validation than line coverage.

Conclusions

The unit testing suite implemented covers the critical paths of FaceAttendEDU, especially in the security and biometrics modules. Severe risks such as identity impersonation and privilege escalation have been successfully mitigated. Remaining gaps lie in the integration with external services (Kafka), which will be covered in the next phase of integration testing.

Author's Note. Diego Andres Gutierrez Nuñez is responsible for the execution and validation of these tests.

References

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

ISO/IEC/IEEE 29148:2018. *Systems and software engineering — Life cycle processes — Requirements engineering*.

IEEE 829. *Standard for Software and System Test Documentation*.